Installation and Usage Guide
IIC-OSIC-TOOLS



1 Introduction

IIC-OSIC-TOOLS is a collection of open-source tools for designing and testing integrated circuits. Everything is packaged inside a Docker container, so you do not need to install the individual tools on your own machine. The full list of tools (Section 6) and further documentation is available in the project's Git repository [1].

2 Installation

2.1 Installing Docker

Docker runs IIC-OSIC-TOOLS in an isolated environment, which reduces the risk of affecting your host operating system.

2.1.1 Windows

1. Download Docker Desktop from the official website [2].
2. Run the installer and follow the instructions. Enable WSL 2 if prompted.
3. Launch Docker Desktop and wait until it reports that Docker is running.
4. Optionally, verify the installation by running the following command in Command Prompt or PowerShell:

```
docker --version
```

2.1.2 Linux and macOS

1. Install Docker according to your operating system:
   - **Linux:** Follow the Docker Engine installation guide [3] for your distribution.
   - **macOS:** Follow the Docker Desktop installation guide [4] and choose the installer that matches your Mac's architecture.
2. Make sure Docker is running.
   - **Linux:** Start the Docker service if it is not already active.
   - **macOS:** Launch Docker Desktop.
3. (Optional) Verify the installation:

```
docker --version
```

Tip

Linux: You need to add your user to the `docker` group and log out and back in (see the Docker post-installation steps [5]).
Linux and macOS: Do not run our startup script (provided later in Section 3.1) with **sudo**, as this may later cause permission problems with the `~/eda/designs` folder in our container.

3 Running IIC-OSIC-TOOLS

3.1 Starting the Container

Important

When you run IIC-OSIC-TOOLS for the first time, the Docker image is downloaded automatically from Docker Hub. This can take several minutes, depending on your internet speed. The image also needs disk space for storage and extraction, so make sure you have **at least 20 GB** free before you start.

Windows.
Download the script from the following link: https://github.com/Stralax/iic-osic-tools/releases/download/v1.0/start_iic.bat. Make sure Docker Desktop is running. Then either double-click the downloaded `start_iic.bat` file, or open the directory where you downloaded it in a terminal and run the following command:

```
.\start_iic.bat
```

Linux and macOS.
Download the script from the following link: https://github.com/Stralax/iic-osic-tools/releases/download/v1.0/start_iic.sh. Make sure Docker is running, then open a terminal in the directory containing the downloaded script.

Make the script executable (only needed once):

```
chmod +x start_iic.sh
```

Then run the startup script:

```
./start_iic.sh
```

Tip

macOS: If the image download fails due to lack of space even though your Mac has enough free storage, increase the virtual disk limit in Docker Desktop: **Settings** → **Resources** → **Disk image size** (at least **40 GB**), then click **Apply & restart**.

3.2 Opening the Desktop in the Browser

Once the container is running, open your browser and enter this address:

```
http://localhost/vnc.html?password=abc123
```

You now have full access to the desktop environment of the container.

Tip

In this guide, we provide our customized container, which is based on the official IIC-OSIC-TOOLS image [1], so everything from the standard version is still included. The differences are the following **additions**:

- **PyOPUS GUI** based on PySide6, with support for parallel execution,
- **Visual Studio Code** editor,
- **SpiceOpus** (amd64 only, not available on Apple silicon Macs).

For more information and details, see the Stralax/iic-osic-tools Git repository [6].

4 Using IIC-OSIC-TOOLS

4.1 Working with Files

The container has a directory that is shared with your local machine. This directory is located at `/foss/designs` in the container and corresponds to `~/eda/designs` on your local machine.

You can share files between your local machine and the container by placing them in the shared directory. Any changes made to these files in either environment are automatically reflected in the other. The absolute path on your local machine for `~/eda/designs` folder is:

Windows:

```
C:\Users\%USERNAME%\eda\designs
```

Linux:

```
/home/${USER}/eda/designs
```

macOS:

```
/Users/${USER}/eda/designs
```

Important

The directory `~/eda/designs` is **persistent**: its contents are preserved when the container is reset. Anything installed or changed elsewhere in the container (for example additional libraries) may be **lost** on reset. Always store your projects, scripts and other important files in `~/eda/designs`.

4.2 Resetting and Updating the Container

If something goes wrong or you want a fresh environment, run the startup script (`start_iic.sh` or `start_iic.bat`) again. If the container already exists, the script asks what to do: press `"s"` to start it (or stop it, if it is running) and `"r"` to remove it. After removing it, run the script once more to create a fresh container.

Each time the script is started, it also checks whether a newer version of the image is available. If there is one, you can press `"u"` to update, which removes the old container and creates a new one from the new image. Any other key starts the old container without updating.

Important

Files stored in `~/eda/designs` remain available after a reset or an update. Everything stored elsewhere in the container will be **lost**.

5 Installed PDKs

All installed PDKs are located in the container's `/foss/pdks` directory. To switch PDKs in the container, use the command '`sak-pdk`'. When called without arguments, it displays a list of installed PDKs. For example, to switch to IHP, enter:

```
sak-pdk ihp-sg13g2
```

or to switch to sky130A enter:

```
sak-pdk sky130A
```

6 Available Built-in Tools

Tool	Description	Tool	Description
`abc`	Sequential logic synthesis and formal verification	`openems`	Electromagnetic field solver using the EC-FDTD method
`amaranth`	A Python-based HDL tool chain	`openram`	OpenRAM Python library
`cace`	A Python-based circuit automatic characterization engine	`openroad`	RTL2GDS engine used by librelane
`charlib`	A characterization library for standard cells	`opensta`	Gate-level static timing verifier
`chipify`	GUI and CLI wrapper for mismatch simulation, parameter sweeps, and yield analysis with xschem and ngspice	`openvaf-reloaded`	Verilog-A compiler for device models
`ciel`	Version manager (and builder) for open-source PDKs	`osic-multitool`	Collection of useful scripts and documentation
`cocotb`	Simulation library for writing VHDL and Verilog test benches in Python	`padring`	Padring generation tool
`covered`	Verilog code coverage	`palace`	3D finite element solver for computational electromagnetics
`cvc`	Circuit validity checker (ERC)	`prjtrellis`	Bitstream tools and database for Lattice ECP5 FPGAs, providing ecppack
`edalize`	Python abstraction library for EDA tools	`pulp-tools`	PULP platform tools consisting of bender, verible, and sv2v
`fault`	Design-for-testing (DFT) solution	`pygmid`	Python version of the gm/Id starter kit from Boris Murmann
`fusesoc`	Package manager and build tools for SoC	`pyopus`	Simulation runner and optimization tool for analog circuits
`gaw3-xschem`	Waveform plot tool for xschem	`pyrtl`	Collection of classes for Pythonic RTL design
`gds2palace/ setupEM`	Setup tools for Palace EM simulation	`pyspice`	Interface ngspice and Xyce from Python
`gds3d`	A 3D viewer for GDS files	`pyuvm`	Universal Verification Methodology implemented in Python (instead of SystemVerilog) using cocotb
`gdscheck`	Rust DRC engine for GDSII layouts, with rules in YAML decks and a KLayout-compatible report database as output (IHP SG13G2 and SG13CMOS5L)	`pyverilog`	Python toolkit for Verilog
`gdsfactory`	Python library for GDS generation	`qflow`	Collection of useful conversion tools
`gdsfill`	Rust tool for inserting dummy metal fill into semiconductor layouts	`qucs-s`	Simulation environment with RF emphasis
`gdspy`	Python module for the creation and manipulation of GDS files	`rggen`	Code generation tool for control and status registers
`gf180mcu`	GlobalFoundries 180 nm CMOS PDK	`risc-v toolchain`	GNU compiler toolchain for RISC-V cores

Tool	Description	Tool	Description
ghdl-yosys-plugin	VHDL plugin for yosys	riscv-pk	RISC-V proxy kernel and bootloader
ghdl	VHDL simulator	schemdraw	Python package for drawing electrical schematics
gmsh	Three-dimensional finite element mesh generator	siliconcompiler	Modular build system for hardware
gtkwave	Waveform plot tool for digital simulation	sky130	SkyWater Technologies 130 nm CMOS PDK
hdl21	Analog hardware description library	slang yosys plugin	Slang-based plugin for yosys for SystemVerilog support
icestorm	Bitstream tools for Lattice iCE40 FPGAs, providing icepack and iceprog	slang	SystemVerilog parsing and translation (e.g. to Verilog)
ihp-sg13g2	IHP Microelectronics 130 nm SiGe:C BiCMOS PDK (partial PDK, not fully supported yet; xschem and ngspice simulation works incl. PSP MOSFET model)	snp2le	Converts Touchstone S-parameter files into lumped-element netlists for ngspice and VACASK
ihp-sg13cmos5l	IHP Microelectronics 130 nm CMOS PDK (M1-M4-TM1 metal stack), shipped in the same repository as ihp-sg13g2	spicebind	Lightweight bridge enabling co-simulation of analog ngspice circuits alongside HDL simulators
irsim	Switch-level digital simulator	spicelib	Library to interact with SPICE-like simulators
iverilog	Verilog simulator	spike	Spike RISC-V ISA simulator
kactus2	Graphical editor for IP-XACT files, which are used to describe hardware components and their interfaces	spyci	Analyze/plot ngspice/Xyce output data with Python
kepler-formal	Logic equivalence checking (LEC) tool for OpenROAD	surelog	SystemVerilog parser, elaborator, and UHDM compiler
klayout	Layout viewer and editor for GDS and OASIS, with packages: klayout-pex, klive, xsection, and klayout-productivity-suite with various iic-jku KLayout plugins	surfer	Waveform viewer with snappy usable interface and extensibility
lctime	Characterization kit for CMOS cells	svck	Customizable SystemVerilog linter (uses verible)
libman	Design library manager to manage cells and views	vacask	A modern Verilog-A based analog circuit simulator
librelane	Successor of OpenLane(2), RTL2GDS flow scripts	verilator	Fast Verilog simulator
magic	Layout editor with DRC and PEX	verible	SystemVerilog parser, style-linter, formatter, and language server
najaeda	Data structures and APIs for the development of post logic synthesis EDA algorithms	veryl	A modern hardware description language, based on SystemVerilog
netgen	Netlist comparison (LVS)	vlog2verilog	Verilog file conversion
nextpnr	FPGA place-and-route for Lattice iCE40 (nextpnr-ice40) and Lattice ECP5 (nextpnr-ecp5)	vlsirtools	Interchange formats for chip design
ngspice	SPICE analog and mixed-signal simulator, with OSDI support	xcircuit	Schematic editor
ngspyce	Python bindings for ngspice	xschem	Schematic editor
nvc	VHDL simulator and compiler	xyce	Fast parallel SPICE simulator
open_pdks	PDK setup scripts	yosys	Verilog synthesis tool (with GHDL plugin for VHDL synthesis and Slang plugin for SystemVerilog synthesis), incl. eqy (equivalence checker), sby (formal verification), and mcy (mutation coverage)
opencdc	Static clock domain crossing (CDC) checker for Verilog/SystemVerilog RTL	RF toolkit	FastHenry2, FasterCap, openEMS, and scikit-rf

References

[1] IIC-OSIC-TOOLS Git repository. https://github.com/iic-jku/iic-osic-tools

[2] Docker Desktop. https://www.docker.com/products/docker-desktop/

[3] Docker Engine installation guide Linux. https://docs.docker.com/engine/install/

[4] Docker Desktop installation guide macOS. https://docs.docker.com/desktop/setup/install/mac-install/

[5] Docker post-installation steps https://docs.docker.com/engine/install/linux-postinstall/

[6] Custom IIC-OSIC-TOOLS. https://github.com/Stralax/iic-osic-tools